

AI ASSISTED CODING

ASSIGNMENT 18.2

NAME: KADARLA HASINI

HT NO: 2403A52062

BATCH NO: 01

TASK 1~

Connect to a Movie Database API (e.g., OMDb or TMDB) to fetch details of a movie.

Instructions:

Prompt AI to generate Python code to query the API by movie title.

Handle errors like invalid movie name, missing/expired API key, and timeout.

Display title, release year, genre, IMDb rating, and director

PROMPT:

Generate a Python script that connects to a Movie Database API (like OMDb or TMDB) to fetch movie details by title. The script should handle errors such as invalid movie name, missing or expired API key, and

request timeout. It should display the movie's title, release year, genre, IMDb rating, and director in a clean, formatted output

CODE & OUTPUT:

```
import requests

def get_movie_details(movie_title, api_key):
    """
    Fetch and display movie details from OMDb API.
    """
    base_url = "https://www.omdbapi.com/"
    params = {
        "t": movie_title,
        "apikey": api_key
    }

    try:
        response = requests.get(base_url, params=params, timeout=5)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

        data = response.json()

        # Check for invalid title or API key error reported by OMDb
        if data.get("Response") == "False":
            print(f"❌ Error from OMDb API: {data.get('Error')}")
            return

        # Display movie details
        print("\n🎬 Movie Details:")
        print("-" * 40)
        print(f"Title       : {data.get('Title', 'N/A')}")
        print(f"Year        : {data.get('Year', 'N/A')}")
        print(f"Genre       : {data.get('Genre', 'N/A')}")
        print(f"IMDb Rating : {data.get('imdbRating', 'N/A')}")
        print(f"Director    : {data.get('Director', 'N/A')}")
        print("-" * 40)
```

```
except requests.exceptions.Timeout:
    print("⌚ Error: Request timed out. Please check your internet connection and try again.")
except requests.exceptions.HTTPError as http_err:
    print(f"⚠ HTTP Error: {http_err}. This could be an invalid API key or a server issue.")
except requests.exceptions.RequestException as err:
    print(f"💥 An unexpected request error occurred: {err}")
except ValueError as json_err:
    print(f"❌ Error decoding JSON response: {json_err}. The API might have sent malformed data.")
```

```
# -----
# Example usage
# -----
if __name__ == "__main__":
    # Get a free API key from https://www.omdbapi.com/apikey.aspx
    # Replace "YOUR_OMDB_API_KEY" with your actual key
    api_key = "YOUR_OMDB_API_KEY"
    movie_title = "Inception"

    get_movie_details(movie_title, api_key)
```

... ⚠ HTTP Error: 401 Client Error: Unauthorized for url: https://www.omdbapi.com/?t=Inception&apikey=YOUR_OMDB_API_KEY.

CODE EXPLANATION:

This Python script uses the OMDb API to fetch movie details by title. It handles errors such as invalid movie names, incorrect API keys, and request timeouts. The code then displays key information like the movie's title, release year, genre, IMDb rating, and director, ensuring robust interaction with the API.

TASK 2~

Use a Public Transport API (e.g., city bus/train API or mock data) to fetch live arrival times.

Instructions:

- Fetch the next 5 arrivals for a given stop/station ID.
- Handle invalid station codes, unavailable service, and malformed responses.
- Display results in a readable table with route number, destination, and arrival time.

PROMPT:

Generate a Python script that connects to a Public Transport API (like a city bus/train API or mock data) to fetch live arrival times. The script should get the next 5 arrivals for a given stop/station ID, handle invalid station codes, unavailable services, and malformed responses. Display the results in a clear table with route number, destination, and arrival time.

CODE:

```
import pandas as pd
import datetime
import time
import json

# --- Mock API Data (simulates a public transport API) ---
```

```

MOCK_API_DATA = {
    "station_A": {
        "status": "success",
        "arrivals": [
            {"route": "101", "destination": "Downtown", "time": (datetime.datetime.now() + datetime.timedelta(minutes=5)).isoformat() + "Z"},
            {"route": "202", "destination": "Uptown", "time": (datetime.datetime.now() + datetime.timedelta(minutes=12)).isoformat() + "Z"},
            {"route": "101", "destination": "Downtown", "time": (datetime.datetime.now() + datetime.timedelta(minutes=20)).isoformat() + "Z"},
            {"route": "303", "destination": "Airport", "time": (datetime.datetime.now() + datetime.timedelta(minutes=25)).isoformat() + "Z"},
            {"route": "202", "destination": "Uptown", "time": (datetime.datetime.now() + datetime.timedelta(minutes=30)).isoformat() + "Z"},
            {"route": "101", "destination": "Downtown", "time": (datetime.datetime.now() + datetime.timedelta(minutes=45)).isoformat() + "Z"}, #
            More than 5 for filtering
        ]
    },
    "station_B": {
        "status": "success",
        "arrivals": [
            {"route": "505", "destination": "Suburban", "time": (datetime.datetime.now() + datetime.timedelta(minutes=2)).isoformat() + "Z"},
            {"route": "606", "destination": "City Center", "time": (datetime.datetime.now() + datetime.timedelta(minutes=10)).isoformat() + "Z"},
            {"route": "505", "destination": "Suburban", "time": (datetime.datetime.now() + datetime.timedelta(minutes=20)).isoformat() + "Z"},
        ]
    },
    "invalid_station": {
        "status": "error",
        "message": "Invalid station ID provided"
    },
    "no_service": {
        "status": "success",
        "arrivals": []
    },
    "malformed_response": "this is not json and will cause an error"
}

```

```

class MockAPIError(Exception):
    """Custom exception for mock API errors."""
    pass

def _mock_api_call(station_id):
    """Simulates an API call to a public transport service."""
    print(f"Simulating API call for station ID: {station_id}")
    time.sleep(0.5) # Simulate network latency

    response_data = MOCK_API_DATA.get(station_id)

    if response_data is None:
        # Simulate a 404 or unknown error for completely non-existent mock IDs
        raise MockAPIError(f"API Error: Station '{station_id}' not found in mock data. (Simulated 404)")

    # Simulate malformed JSON response
    if isinstance(response_data, str):
        print(f"✗ Simulated malformed response for '{station_id}'.")
        raise json.JSONDecodeError("Expecting value", response_data, 0)

    # Simulate API-level error (e.g., invalid ID explicitly in data)
    if response_data.get("status") == "error":
        print(f"✗ API reported error for '{station_id}': {response_data.get('message', 'Unknown API error')}")
        raise MockAPIError(response_data.get('message', 'Unknown API error'))

```

```

return response_data

def get_public_transport_arrivals(station_id):
    """
    Fetches and displays the next 5 public transport arrivals for a given station ID.
    Handles invalid station codes, unavailable service, and malformed responses.
    """
    print(f"\n--- Fetching arrivals for Station ID: {station_id} ---")
    try:
        # Replace _mock_api_call with actual requests.get().json() for a real API
        # Example: response = requests.get(f"YOUR_API_URL/arrivals?station_id={station_id}&api_key=YOUR_KEY")
        # data = response.json()
        data = _mock_api_call(station_id)

        arrivals = data.get('arrivals', [])

        if not arrivals:
            print(f"❗ No upcoming arrivals found for Station ID: {station_id}.")
            return

        processed_arrivals = []
        now = datetime.datetime.now(datetime.timezone.utc)

        for arrival in arrivals:
            try:
                arrival_time_str = arrival.get('time')
            except:
                print(f"⚠ Warning: Arrival record missing time: {arrival}")
                continue

            # Ensure timezone awareness for comparison
            arrival_dt = datetime.datetime.fromisoformat(arrival_time_str.replace('Z', '+00:00'))

            if arrival_dt > now:
                time_until = arrival_dt - now
                minutes_until = int(time_until.total_seconds() / 60)
                processed_arrivals.append({
                    'Route': arrival.get('route', 'N/A'),
                    'Destination': arrival.get('destination', 'N/A'),
                    'Arrival Time': arrival_dt.strftime('%H:%M'),
                    'Time Until (min)': minutes_until
                })
            except ValueError as e:
                print(f"⚠ Warning: Could not parse arrival time '{arrival.get('time')}': {e}")
            except Exception as e:
                print(f"⚠ Warning: Error processing arrival record {arrival}: {e}")

        if not processed_arrivals:
            print(f"❗ No upcoming arrivals found after filtering past times for Station ID: {station_id}.")
            return

        # Sort by 'Time Until (min)' and get the next 5
        df = pd.DataFrame(processed_arrivals)
        df = df.sort_values(by="Time Until (min)").head(5)

        print(f"\n✅ Next 5 Arrivals:")
        print(df.to_string(index=False))

```

```

except MockAPIError as e:
    print(f'❌ Public Transport Service Error: {e}')
except json.JSONDecodeError:
    print("❌ Malformed response from API: Expected JSON but received invalid data.")
except Exception as e:
    print(f'❌ An unexpected error occurred: {e}')

# --- Example Usage ---
if __name__ == "__main__":
    # Test cases
    get_public_transport_arrivals("station_A")
    get_public_transport_arrivals("station_B")
    get_public_transport_arrivals("no_service")
    get_public_transport_arrivals("invalid_station")
    get_public_transport_arrivals("malformed_response")
    get_public_transport_arrivals("non_existent_station_in_mock_data")

```

OUTPUT:

```

--- Fetching arrivals for Station ID: station_A ---
Simulating API call for station ID: station_A
...
✅ Next 5 Arrivals:
Route Destination Arrival Time Time Until (min)
101 Downtown 08:26 4
202 Uptown 08:33 11
101 Downtown 08:41 19
303 Airport 08:46 24
202 Uptown 08:51 29

--- Fetching arrivals for Station ID: station_B ---
Simulating API call for station ID: station_B
...
✅ Next 5 Arrivals:
Route Destination Arrival Time Time Until (min)
505 Suburban 08:23 1
606 City Center 08:31 9
505 Suburban 08:41 19

--- Fetching arrivals for Station ID: no_service ---
Simulating API call for station ID: no_service
🚫 No upcoming arrivals found for Station ID: no_service.

--- Fetching arrivals for Station ID: invalid_station ---
Simulating API call for station ID: invalid_station
❌ API reported error for 'invalid_station': Invalid station ID provided
❌ Public Transport Service Error: Invalid station ID provided

--- Fetching arrivals for Station ID: malformed_response ---
Simulating API call for station ID: malformed_response
❌ Simulated malformed response for 'malformed_response'.
❌ Malformed response from API: Expected JSON but received invalid data.

--- Fetching arrivals for Station ID: non_existent_station_in_mock_data ---
Simulating API call for station ID: non_existent_station_in_mock_data
❌ Public Transport Service Error: API Error: Station 'non_existent_station_in_mock_data' not found in mock data. (Simulated 404)

```

CODE EXPLANATION:

This Python script simulates a public transport API to fetch and display the next five arrival times for a given station ID. It uses mock data and robustly handles various scenarios, including valid station data, no service, invalid station codes, malformed API responses, and non-existent stations, presenting results in a readable table. It demonstrates comprehensive error management.

TASK 3~

Connect to a stock data API (e.g., Alpha Vantage, Yahoo Finance) to fetch daily stock prices.

Instructions:

Prompt AI to generate Python function to query stock data by ticker symbol.

Handle API call limits, invalid ticker symbols, and null responses.

Display opening price, closing price, high, low, and trading volume

PROMPT:

Generate a Python script that connects to a Stock Data API (such as Alpha Vantage or Yahoo Finance) to fetch daily stock prices. The script should query data using a ticker symbol entered by the user. It must handle API call limits, invalid ticker symbols, and null or missing responses. Display the stock's opening price, closing price, highest and lowest prices, and trading volume in a clear format

CODE:

```
import requests
import json

def get_daily_stock_data(ticker_symbol, api_key):
    """
    Fetches and displays daily stock prices for a given ticker symbol from Alpha Vantage.
    Handles API call limits, invalid ticker symbols, and null/error responses.
    """
    base_url = "https://www.alphavantage.co/query"
    params = {
        "function": "TIME_SERIES_DAILY",
        "symbol": ticker_symbol,
        "apikey": api_key,
        "outputsize": "compact" # compact returns only the latest 100 data points
    }

    print(f"\n--- Fetching daily stock data for: {ticker_symbol} ---")
    try:
        response = requests.get(base_url, params=params, timeout=10)
        response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

    data = response.json()

    # Alpha Vantage often returns an 'Error Message' or 'Note' key for issues
    if "Error Message" in data:
        print(f"❌ Alpha Vantage API Error for {ticker_symbol}: {data['Error Message']}")
        return
    if "Note" in data:
```

```

print(f"⚠️ Alpha Vantage API Note for {ticker_symbol}: {data['Note']}\n This often indicates hitting the API call limit (e.g., 5 calls per minute for free tier).")
return
if not data or "Time Series (Daily)" not in data:
    print(f"❌ No daily time series data found for ticker: {ticker_symbol}. It might be an invalid symbol or no data is available.")
    return

time_series = data["Time Series (Daily)"]
if not time_series:
    print(f"❌ No daily time series data found for ticker: {ticker_symbol}. It might be an invalid symbol or no data is available.")
    return

# Get the latest trading day's data
latest_date = list(time_series.keys())[0]
latest_data = time_series[latest_date]

print(f"✅ Latest data for {ticker_symbol} on {latest_date}:")
print("-" * 40)
print(f"Open : {latest_data.get('1. open', 'N/A')}")
print(f"High : {latest_data.get('2. high', 'N/A')}")
print(f"Low : {latest_data.get('3. low', 'N/A')}")
print(f"Close : {latest_data.get('4. close', 'N/A')}")
print(f"Volume : {latest_data.get('5. volume', 'N/A')}")
print("-" * 40)

except requests.exceptions.Timeout:
    print(f"⌚ Error: Request for {ticker_symbol} timed out. Please check your internet connection and try again.")
except requests.exceptions.HTTPError as http_err:
    print(f"⚠️ HTTP Error for {ticker_symbol}: {http_err}. This could be an API server issue.")
except requests.exceptions.RequestException as err:
    print(f"🚫 An unexpected request error occurred for {ticker_symbol}: {err}")
except json.JSONDecodeError:
    print(f"❌ Error decoding JSON response for {ticker_symbol}. The API might have sent malformed data.")
except Exception as e:
    print(f"❌ An unexpected error occurred for {ticker_symbol}: {e}")

# -----
# Example usage
# -----
if __name__ == "__main__":
    # Get a free API key from https://www.alphavantage.co/support/#api-key
    # Replace "YOUR_ALPHA_VANTAGE_API_KEY" with your actual key
    alpha_vantage_api_key = "YOUR_ALPHA_VANTAGE_API_KEY"

    # Test cases
    get_daily_stock_data("IBM", alpha_vantage_api_key)
    get_daily_stock_data("GOOGL", alpha_vantage_api_key)
    get_daily_stock_data("INVALIDSTOCK", alpha_vantage_api_key) # Invalid ticker
    # Add a delay if testing multiple calls with a free API key to avoid rate limits
    # time.sleep(15)
    # get_daily_stock_data("MSFT", alpha_vantage_api_key)

```


OUTPUT:

```
...
--- Fetching daily stock data for: IBM ---
✅ Latest data for IBM on 2025-11-10:
-----
Open       : 306.8200
High       : 309.9400
Low        : 304.2300
Close      : 309.1300
Volume     : 2930886
-----

--- Fetching daily stock data for: GOOGL ---
✅ Latest data for GOOGL on 2025-11-10:
-----
Open       : 284.4200
High       : 290.8000
Low        : 282.8550
Close      : 290.1000
Volume     : 29557309
-----

--- Fetching daily stock data for: INVALIDSTOCK ---
❌ Alpha Vantage API Error for INVALIDSTOCK: Invalid API call. Please retry or visit the documentation (https://www.alphavantage.co/documentation/) for
```

CODE EXPLANATION:

This script connects to the Alpha Vantage API to fetch daily stock data for given ticker symbols. It retrieves opening/closing prices, high, low, and volume. The code includes robust error handling for invalid API keys, incorrect ticker symbols, and API call limits, providing clear messages for different failure scenarios.

TASK 4~

Real-Time Application: Translation API

Scenario: Build a translator using a free Translation API (e.g., Libre Translate, Google Translate).

Requirements:

Accept input text and target language from the user.

Handle invalid language codes, API quota exceeded, and empty text input.

Display original and translated text clearly.

Implement a retry mechanism if the API fails on the first attempt.

PROMPT:

Generate a Python script using a free Translation API (like LibreTranslate) to build a real-time translator. The script should accept input text and target language from the user, handle invalid language codes, API quota exceeded errors, and empty input text. It must retry once if the API fails, and clearly display both the original and translated text

CODE:

```
import requests
import json
import time

# --- LibreTranslate API Configuration ---
# You can use the public instance (libretranslate.com) or self-host one.
# The public instance might have rate limits or downtime. For production, self-hosting is recommended.
LIBRE_TRANSLATE_API_URL = "https://libretranslate.com/translate"

def translate_text(text, target_language_code, source_language_code="auto", max_retries=3,
retry_delay_seconds=2):
    """
    Translates text using the LibreTranslate API with a retry mechanism.

    Args:
    text (str): The text to translate.
    target_language_code (str): The target language code (e.g., 'es', 'fr', 'de').
    source_language_code (str): The source language code (e.g., 'en', 'auto'). Defaults to 'auto'.
    max_retries (int): Maximum number of retries for API calls.
    retry_delay_seconds (int): Delay between retries in seconds.

    Returns:
    str: The translated text, or an error message if translation fails.
    """
    if not text.strip():
        return "❌ Error: Input text cannot be empty."

    print(f"\n--- Attempting to translate to {target_language_code.upper()} ---")

    payload = {
        "q": text,
```

```

"source": source_language_code,
"target": target_language_code,
"format": "text"
}
headers = {
"Content-Type": "application/json"
}

for attempt in range(1, max_retries + 1):
try:
print(f"Attempt {attempt}/{max_retries}...")
response = requests.post(LIBRE_TRANSLATE_API_URL, data=json.dumps(payload), headers=headers,
timeout=10)
response.raise_for_status() # Raise HTTPError for bad responses (4xx or 5xx)

data = response.json()

if 'error' in data:
# LibreTranslate specific error handling
error_message = data.get('error', 'Unknown API error')
print(f"❌ API Error (Attempt {attempt}): {error_message}")
# For certain errors, we might not want to retry, e.g., invalid language
if "Invalid target language" in error_message or "Invalid source language" in error_message:
return f"❌ Translation failed: {error_message}"
if attempt < max_retries:
print(f"🔄 Retrying in {retry_delay_seconds} seconds...")
time.sleep(retry_delay_seconds)
continue
return f"❌ Translation failed after {max_retries} attempts: {error_message}"

return data.get('translatedText', 'Translation not available.')

except requests.exceptions.Timeout:
print(f"⌚ Request timed out (Attempt {attempt}).")
if attempt < max_retries:
print(f"🔄 Retrying in {retry_delay_seconds} seconds...")
time.sleep(retry_delay_seconds)
continue
return f"❌ Translation failed after {max_retries} attempts: Request timed out."
except requests.exceptions.HTTPError as http_err:
print(f"⚠ HTTP Error (Attempt {attempt}): {http_err}")
if http_err.response.status_code == 429: # Too Many Requests

```

```

print("API quota might be exceeded or rate limit hit.")
if attempt < max_retries:
    print(f"Retrying in {retry_delay_seconds} seconds...")
    time.sleep(retry_delay_seconds)
    continue
return f"❌ Translation failed after {max_retries} attempts: HTTP Error: {http_err}"
except requests.exceptions.RequestException as req_err:
    print(f"❌ Connection Error (Attempt {attempt}): {req_err}")
    if attempt < max_retries:
        print(f"Retrying in {retry_delay_seconds} seconds...")
        time.sleep(retry_delay_seconds)
        continue
    return f"❌ Translation failed after {max_retries} attempts: Connection error: {req_err}"
except json.JSONDecodeError:
    print(f"❌ JSON Decode Error (Attempt {attempt}): Malformed response from API.")
    if attempt < max_retries:
        print(f"Retrying in {retry_delay_seconds} seconds...")
        time.sleep(retry_delay_seconds)
        continue
    return f"❌ Translation failed after {max_retries} attempts: Malformed API response."
except Exception as e:
    print(f"❌ An unexpected error occurred (Attempt {attempt}): {e}")
    return f"❌ An unexpected error occurred: {e}"


if __name__ == "__main__":
    print("### Interactive Translation Tool ###")
    print("Example language codes: en=English, es=Spanish, fr=French, hi=Hindi, de=German, ta=Tamil")

    while True:
        user_text = input("\nEnter text to translate (or type 'quit' to exit): ")
        if user_text.lower() == 'quit':
            break

    user_target_lang = input("Enter target language code: ").strip().lower()

    translated_output = translate_text(user_text, user_target_lang)
    print("-----")
    print(f"Original Text : {user_text}")
    print(f"Translated Text: {translated_output}")
    print("-----")

```

OUTPUT:

```
... ### Interactive Translation Tool ###
Example language codes: en=English, es=Spanish, fr=French, hi=Hindi, de=German, ta=Tamil

Enter text to translate (or type 'quit' to exit): how are you
Enter target language code: fr

--- Attempting to translate to FR ---
Attempt 1/3...
⚠ HTTP Error (Attempt 1): 400 Client Error: Bad Request for url: https://libretranslate.com/translate
Retrying in 2 seconds...
Attempt 2/3...
⚠ HTTP Error (Attempt 2): 400 Client Error: Bad Request for url: https://libretranslate.com/translate
Retrying in 2 seconds...
Attempt 3/3...
⚠ HTTP Error (Attempt 3): 400 Client Error: Bad Request for url: https://libretranslate.com/translate
=====
Original Text : how are you
Translated Text: ❌ Translation failed after 3 attempts: HTTP Error: 400 Client Error: Bad Request for url: https://libretranslate.com/translate
=====

Enter text to translate (or type 'quit' to exit): quit
```

CODE EXPLANATION:

This Python script creates a translation tool using the LibreTranslate API. It handles user input for text and target language. The function incorporates robust error handling for issues like invalid language codes, empty text, network problems, and API errors (e.g., quota exceeded). A retry mechanism is implemented to reattempt failed API calls, ensuring resilient translation and clear error reporting.